



hexagone

École Supérieure d'Informatique

Programmation orientée objet

4PYTH – Langage Python

Chris CHEVALIER



hexagone

École Supérieure d'Informatique

Sommaire

- #1** Pourquoi des objets ?
- #2** Classes et objets
- #3** Encapsulation
- #4** Héritage et polymorphisme
- #5** Composition



#1

Pourquoi des objets ?

Programmation orientée objet



Les limites des dictionnaires

Stockons un compte dans un dictionnaire. Rien ne protège ses données.

```
account.py

user = {"username": "Chris", "role": "analyste",
        "failed_attempts": 0}

user["role"] = "n'importe quoi"    # aucun contrôle
user["failed_attempts"] = 5        # faute de frappe :
                                   # une nouvelle clé !
```

- Les **données** ne sont pas contrôlées
- Les **fonctions** qui les manipulent sont ailleurs, dans un autre fichier
- Rien ne dit quelles clés un compte doit avoir

La programmation orientée objet réunit les **données** et les **comportements** dans un même modèle, qui décide de ce qui est permis.



Le vocabulaire de l'objet

Terme	Définition	Exemple
Classe	Le modèle : il décrit les données et les actions communes	User
Objet (instance)	Un exemplaire concret, créé à partir de la classe	<code>chris</code>
Attribut	Une donnée propre à l'objet : son état	<code>chris.role</code>
Méthode	Une action que l'objet sait faire : une fonction de la classe	<code>chris.record_failure()</code>
Instancier	Créer un objet à partir de sa classe	<code>User("Chris", "analyste")</code>

La classe est le **formulaire vierge** de création de compte ; chaque compte créé à partir de ce formulaire est un **objet**.



#2 Classes et objets

Programmation orientée objet



Définir une classe

```
user.py

class User:
    """Un compte utilisateur."""

    def __init__(self, username, role):
        self.username = username
        self.role = role
        self.failed_attempts = 0
        self.is_locked = False
```

- `class User:` définit la classe, nommée en **PascalCase**
- `__init__` est le **constructeur**. Il est appelé automatiquement à la création de chaque objet
- `self` désigne **l'objet courant**.
`self.role = role` range la valeur dans cet objet
- Un attribut peut avoir une valeur de départ par défaut :
`failed_attempts = 0`



Créer des objets

```
main.py

from user import User

chris = User("Chris", "analyste")
manon = User("Manon", "auditeur")

print(chris.username)      # Chris
print(manon.role)          # auditeur
print(type(chris))         # <class 'user.User'>

chris.failed_attempts = 2
print(manon.failed_attempts) # 0
```

- `from user import User` importe la classe depuis le fichier `user.py`
- Créer un objet à partir d'une classe s'appelle l'**instanciation**. On appelle la classe comme une fonction. Python crée l'objet puis exécute `__init__`
- On accède aux attributs avec le **point** : `chris.username`
- Chaque objet a **ses propres attributs**. Modifier `chris` ne modifie pas `manon`
- Le type de l'objet est sa classe



Un fichier, une responsabilité

```
Organisation du projet

accounts/
├── user.py      # User et Admin
├── server.py    # Server
├── network.py   # Network
└── main.py     # le programme
```

Les fichiers s'écrivent en minuscules (`user.py`) et les classes en PascalCase (`User`), comme le recommande la [PEP 8](#).

On range le code **par domaine**. Une classe et ses classes filles vont dans le même fichier. Le fichier `main.py` utilise les classes sans les définir. On retrouve vite le code et on modifie un domaine sans toucher aux autres.

- **Responsabilité unique** (*Single Responsibility Principle*) : un fichier, une classe ou une fonction n'a qu'une seule raison de changer
- C'est le « S » de SOLID, cinq principes de conception qui rendent le code facile à faire évoluer



Les méthodes

```
user.py

class User:
    def __init__(self, username, role):
        ... # le constructeur déjà vu

    def record_failure(self):
        """Enregistre un échec et verrouille au 3e."""
        self.failed_attempts += 1
        if self.failed_attempts >= 3:
            self.is_locked = True
```

```
main.py

from user import User

chris = User("Chris", "analyste")
for _ in range(3):
    chris.record_failure()
print(chris.is_locked)      # True
```

- Une méthode est une **fonction définie dans la classe**
- Son premier paramètre est toujours `self`. Elle agit sur l'objet qui l'appelle
- `chris.record_failure()` revient à écrire `User.record_failure(chris)`
- La règle de verrouillage est écrite **une seule fois**, au plus près des données



Attributs de classe ou d'instance

```
user.py
class User:
    MAX_ATTEMPTS = 3      # attribut de classe

    def __init__(self, username, role):
        self.username = username  # d'instance
        ...

    def record_failure(self):
        self.failed_attempts += 1
        if self.failed_attempts >= User.MAX_ATTEMPTS:
            self.is_locked = True

main.py
print(User.MAX_ATTEMPTS)      # 3
print(chris.MAX_ATTEMPTS)    # 3 : lu depuis la classe
```

- **Attribut d'instance** (`self.x`) : une valeur par objet
- **Attribut de classe** : une seule valeur, **partagée par tous les objets**
- Le nombre 3 n'est plus répété dans le code. On le modifie à un seul endroit

Idéal pour un **seuil** ou une **capacité maximale** commun à tous les objets d'une classe.



Afficher un objet : `__str__`

Sans `__str__`

```
Terminal
>>> print(chris)
<__main__.User object at 0x10291aa50>
```

Avec `__str__`

```
user.py
class User:
    ...
    def __str__(self):
        state = "actif"
        if self.is_locked:
            state = "verrouillé"
        return f"{self.username} ({self.role}, {state})"
```

```
Terminal
>>> print(chris)
Chris (analyste, verrouillé)
```

- `__str__` est une **méthode spéciale**. `print()` et `str()` l'appellent automatiquement
- Elle **renvoie** une chaîne. Elle ne l'affiche pas
- Les méthodes entourées de `__` (*dunder*) sont réservées à Python : `__init__`, `__str__` ...



Qu'affiche ce programme ?

quiz_1.py

```
class Probe:
    total = 0
    def __init__(self, name):
        self.name = name
        self.hits = 0
        Probe.total += 1
    def hit(self):
        self.hits += 1
        return self.hits

ssh = Probe("ssh")
web = Probe("web")
ssh.hit()
ssh.hit()
web.hit()
print(ssh.hits, web.hits)
print(Probe.total)
print(ssh.hit())
```

2 1 : chaque sonde compte **ses propres** passages

2 : `total` est un attribut de classe, incrémenté à chaque création

3 : `hit()` incrémente puis **renvoie** la nouvelle valeur



#3 Encapsulation

Programmation orientée objet



Le principe et les conventions

L'**encapsulation** consiste à garder la maîtrise des données d'un objet. On les modifie par ses méthodes plutôt que directement.

```
user.py

class User:
    def __init__(self, username, password):
        self.username = username      # public
        self._sessions = []          # usage interne
        self.__password = password    # nom modifié
```

Nom	Signification
username	public
<code>_sessions</code>	« ne pas toucher » : simple convention
<code>__password</code>	Python le renomme en <code>_User__password</code>

En Python, **rien n'est vraiment privé**. Le `_` est un simple signal adressé aux autres développeurs, que Python ne vérifie pas. Le `__` va plus loin. Python renomme l'attribut, ce qui évite de le lire ou de l'écraser **par erreur** depuis l'extérieur ou depuis une classe fille.



Côté sécurité

```
Terminal
>>> chris = User("Chris", "S3cr3t!")
>>> chris.__password
AttributeError: 'User' object has no attribute
'__password'
>>> chris._User__password
'S3cr3t!'
>>> vars(chris)
{'username': 'Chris', '_sessions': [],
 '_User__password': 'S3cr3t!'}
```

- Le double underscore ne fait que **renommer** l'attribut
- `vars()` affiche tous les attributs d'un objet, « privés » compris

L'encapsulation n'est pas un mécanisme de sécurité. Elle protège contre les erreurs, pas contre un attaquant. Un secret ne se stocke jamais en clair. Il se **hache** avec une fonction dédiée comme [bcrypt](#) ou [Argon2](#).



@property : un attribut en lecture seule

```
user.py

class User:
    def __init__(self, username):
        self.username = username
        self._failed_attempts = 0

    @property
    def failed_attempts(self): # lecture seule
        return self._failed_attempts

    def record_failure(self):
        self._failed_attempts += 1
```

```
main.py

chris = User("Chris")
chris.record_failure()
print(chris.failed_attempts) # 1
chris.failed_attempts = 5    # AttributeError
```

- `@property` transforme une méthode en **attribut calculé**. On l'utilise sans parenthèses
- Sans *setter*, l'attribut est en **lecture seule**. Seules les méthodes de la classe, comme `record_failure()`, modifient `_failed_attempts`

C'est ainsi que Python remplace les *getters* et *setters* des autres langages.



@property : valider une écriture

```
user.py

ROLES = ["analyste", "auditeur", "admin"]

class User:
    def __init__(self, username, role):
        self.username = username
        self.role = role          # passe par le setter

    @property
    def role(self):
        return self._role

    @role.setter
    def role(self, value):
        if value not in ROLES:
            raise ValueError(f"rôle inconnu : {value}")
        self._role = value
```

```
main.py

chris = User("Chris", "analyste")
chris.role = "auditeur"  # accepté
chris.role = "hacker"
# ValueError: rôle inconnu : hacker
```

- Le **setter** s'exécute à chaque affectation, y compris dans `__init__`
- Le rôle est validé par **liste blanche**. On n'accepte que les valeurs connues



#4 Héritage et polymorphisme

Programmation orientée objet



Hériter d'une classe

```
user.py (suite)

class Admin(User):
    def __init__(self, username, scope):
        super().__init__(username, "admin")
        self.scope = scope      # attribut en plus
```

```
main.py

from user import User, Admin

root = Admin("root", "serveurs Linux")
print(root.username, root.role)      # root admin
root.record_failure()                # méthode héritée
print(root.failed_attempts)          # 1
print(isinstance(root, User))        # True
```

- `class Admin(User):` signifie que la **classe fille** `Admin` hérite de la **classe mère** `User`
- Elle récupère **tous** ses attributs et **toutes** ses méthodes
- `super().__init__(...)` appelle le constructeur de la classe mère. Sans lui, `username` n'existerait pas

Avant d'hériter, on doit pouvoir dire « un `Admin` **est un** `User` ». Ce test vaut pour toute classe mère et sa classe fille.



Redéfinir une méthode

```
user.py

class User:
    ...
    def describe(self):
        return f"Compte {self.username} ({self.role})"

class Admin(User):
    ...
    def describe(self):          # redéfinition
        return f"Administrateur {self.username}"
```

```
main.py

print(User("Chris", "analyste").describe())
# Compte Chris (analyste)
print(Admin("root", "serveurs Linux").describe())
# Administrateur root
```

- La classe fille peut **remplacer** une méthode héritée en la redéfinissant avec le même nom
- Python utilise toujours la version **la plus proche** de la classe de l'objet

On parle de **redéfinition** (*override*), pas de surcharge. La surcharge (plusieurs méthodes de même nom avec des paramètres différents) **n'existe pas en Python**.



Étendre une méthode avec `super()`

```
user.py (suite)

class Admin(User):
    ...
    def describe(self):
        base = super().describe()    # version de User
        return f"{base} – périmètre : {self.scope}"
```

```
main.py

root = Admin("root", "serveurs Linux")
print(root.describe())
# Compte root (admin) – périmètre : serveurs Linux
```

- `super()` donne accès à la version de la méthode de la **classe mère**
- On **réutilise** son résultat au lieu de le réécrire. Le code n'est pas dupliqué
- Si le format de `User.describe()` change, `Admin` suit automatiquement



Le polymorphisme

```
main.py

from user import User, Admin

accounts = [
    User("Chris", "analyste"),
    Admin("root", "serveurs Linux"),
    User("Manon", "auditeur"),
]
for account in accounts:
    print(account.describe())
```

```
Terminal

Compte Chris (analyste)
Compte root (admin) – périmètre : serveurs Linux
Compte Manon (auditeur)
```

- Un **même appel** produit un comportement **adapté** au type de chaque objet
- La boucle n'a pas besoin de savoir qui est administrateur

Ajouter un nouveau type de compte ne demande **aucune modification** de la boucle.



Tester le type : `isinstance()`

```
main.py (suite)

admins = 0
for account in accounts:
    if isinstance(account, Admin):
        admins += 1
print(admins)                                # 1

print(isinstance(accounts[1], User))          # True
print(type(accounts[1]) == User)             # False
```

- `isinstance(objet, Classe)` est vrai si l'objet est de cette classe **ou d'une classe fille**
- `type(...) == ...` ignore l'héritage. Préférez `isinstance()`
- Pratique pour **compter par type** : administrateurs, mentalistes...

Pratique pour produire des **statistiques par type** d'objet.



Qu'affiche ce programme ?

```
quiz_2.py

class Alert:
    def __init__(self, source):
        self.source = source
    def level(self):
        return "info"
    def report(self):
        return f"[{self.level()}] {self.source}"

class CriticalAlert(Alert):
    def level(self):
        return "CRITIQUE"

alerts = [Alert("ssh"), CriticalAlert("firewall")]
for alert in alerts:
    print(alert.report())
print(isinstance(alerts[1], Alert))
```

[info] ssh : méthodes de
Alert

[CRITIQUE] firewall :
report() est héritée, mais
self.level() appelle la
version **redéfinie**

True : une CriticalAlert est
une Alert



#5 Composition

Programmation orientée objet



« Est un » ou « a un » ?

Relation	Exemple	En Python
Est un → héritage	Un administrateur est un utilisateur	<code>class Admin(User):</code>
A un → composition	Un réseau a des serveurs	<code>self.servers = []</code>

```
server.py

class Server:
    def __init__(self, hostname, ip, services):
        self.hostname = hostname
        self.ip = ip
        self.services = services    # chaînes
```

```
network.py

class Network:
    def __init__(self, name):
        self.name = name
        self.servers = []    # objets Server
```

La **composition** consiste à ranger des objets **dans les attributs** d'un autre objet. Un `Network` n'est pas un `Server`. Il en **contient**. En cas de doute, préférez la composition à l'héritage.



Des objets dans des objets

```
network.py

class Network:
    MAX_SERVERS = 3

    def __init__(self, name):
        self.name = name
        self.servers = []          # objets Server

    def add_server(self, server):
        if len(self.servers) >= Network.MAX_SERVERS:
            raise ValueError("capacité maximale atteinte")
        self.servers.append(server)

    def find_by_ip(self, ip):
        for server in self.servers:
            if server.ip == ip:
                return server
        return None
```

```
main.py

from network import Network
from server import Server

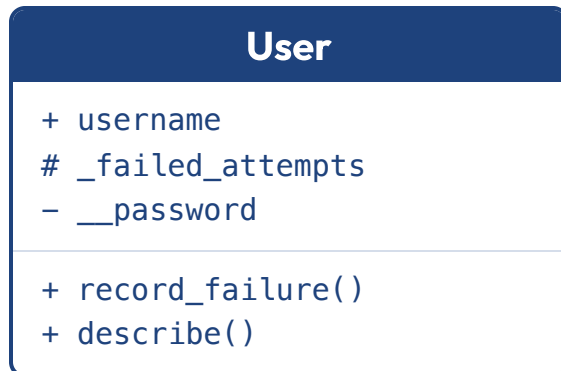
dmz = Network("DMZ")
dmz.add_server(Server("web01",
    "10.0.1.10", ["http", "https"]))
dmz.add_server(Server("mail01",
    "10.0.1.20", ["smtp"]))
dmz.find_by_ip("10.0.1.20").hostname
# 'mail01'
```

- Le réseau **contrôle** ce qu'on lui ajoute, ici sa capacité maximale
- Un dossier contient des fichiers, une équipe contient des membres. La composition est partout



Lire un diagramme de classes UML

L'**UML** (*Unified Modeling Language*) est une notation standard pour dessiner un programme avant de le coder. Son diagramme de classes montre les classes et leurs relations.

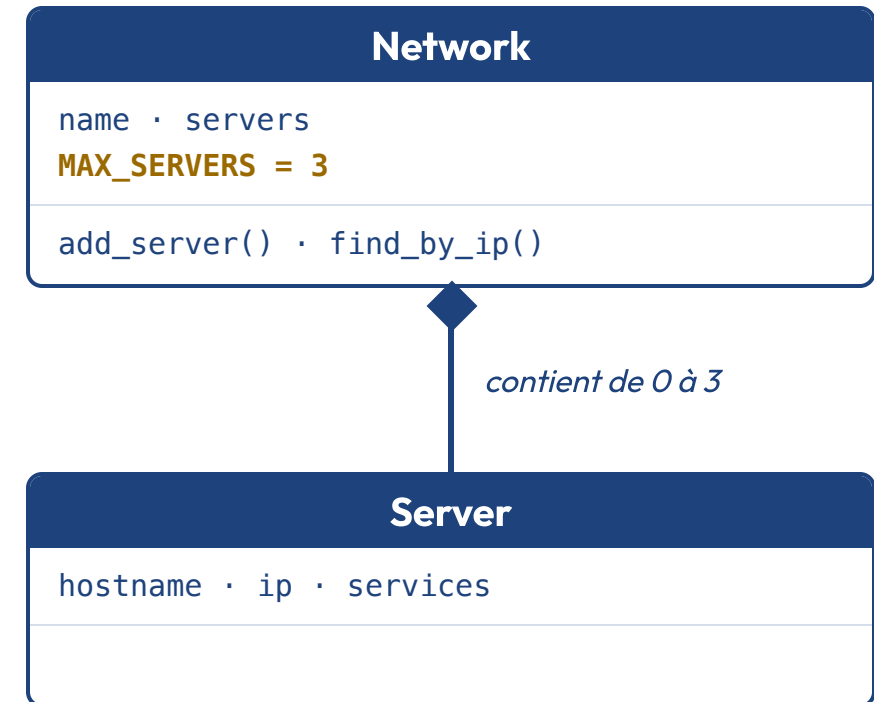
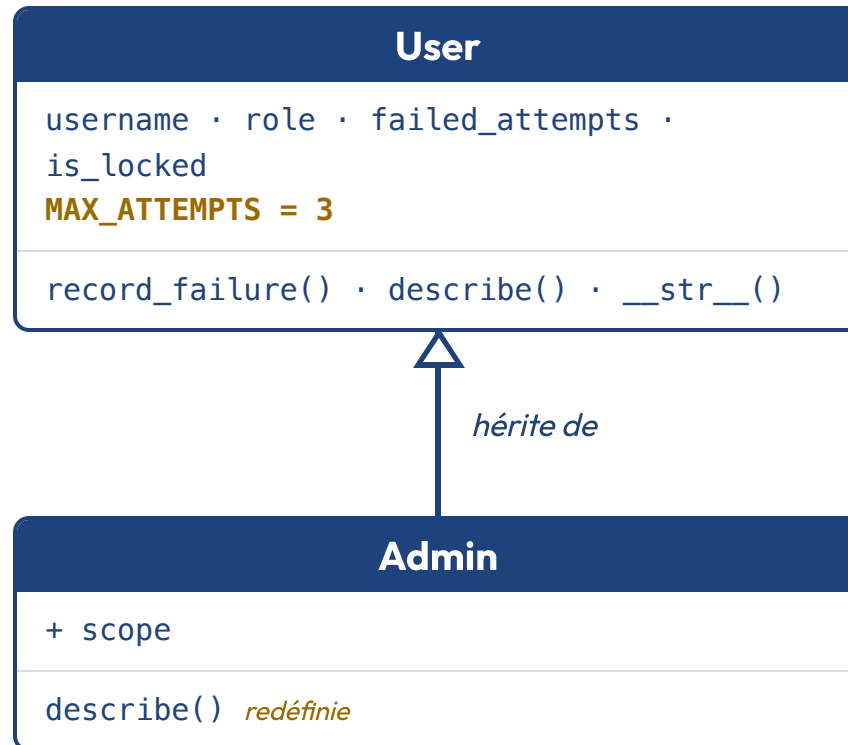


Trois cases : le nom, les attributs puis les méthodes.

Symbole	Signification
+	public : username
#	protégé (classe et classes filles) : <code>_nom</code> en Python
-	privé : en Python, <code>__nom</code>
	héritage : la pointe creuse vise la classe mère
	composition : le losange plein est du côté du conteneur
	agrégation : les objets existent sans le conteneur
0..3 , *	multiplicité : combien d'objets sont liés



Vue d'ensemble





Récapitulatif

Notion	Exemple du cours	À retenir
Classe, <code>__init__</code> , <code>self</code>	User	le modèle ; <code>__init__</code> initialise, <code>self</code> désigne l'objet
Méthode	<code>record_failure()</code>	une action que l'objet sait faire
Attribut de classe	<code>MAX_ATTEMPTS</code> , <code>MAX_SERVERS</code>	partagé par tous les objets de la classe
<code>__str__</code>	<code>print(chris)</code>	rend l'affichage d'un objet lisible
<code>@property</code>	<code>failed_attempts</code> , <code>role</code>	lecture seule ou écriture validée
Héritage et <code>super()</code>	<code>Admin(User)</code>	reprenre une classe puis la compléter
Polymorphisme	<code>describe()</code>	même appel, comportement propre à chaque classe
<code>isinstance()</code>	compter les administrateurs	tester le type d'un objet
Composition	Network → Server	un objet en contient d'autres (« a un »)
Responsabilité unique	<code>user.py</code> , <code>main.py</code>	un fichier par domaine
Diagramme UML	vue d'ensemble	cases, visibilité (+ # -), flèches



Des questions ?

Suite : cours 3 – Fichiers et données, après la pause
Chris CHEVALIER